

Day 13 Learning (27/03/2026)

#Hibernate ORM:

Hibernate is a **Object-Relational Mapping (ORM)** framework that provides a framework for mapping an object-oriented domain model to a relational database. It implements the **JPA (Jakarta Persistence API)** specification, providing a more powerful and developer-friendly layer over traditional JDBC (Java Database Connectivity).

1. The Core Problem:

Before diving into Hibernate, it is essential to understand why it exists. The **Object-Relational Impedance Mismatch** refers to the set of conceptual and technical difficulties that occur when an object-oriented program is stored in a relational database.

- **Granularity:** Your Java model might have more classes than tables in the database.
- **Inheritance:** Databases do not have a natural way to represent inheritance (Class A extending Class B).
- **Identity:** Java uses == or .equals(), while databases use Primary Keys.
- **Associations:** Java uses references (pointers), while databases use Foreign Keys and Join Tables.

2. Architecture Components:

A. Configuration Object

This is the first object you create in a Hibernate application. It is used to specify the properties of the database and the mapping files. It is responsible for creating the **SessionFactory**.

B. SessionFactory (Thread-Safe)

The SessionFactory is a factory for **Session** objects. It is usually created during application startup and kept for the entire duration of the app. It is a "heavyweight" object because it maintains the Second-Level Cache, metadata about mappings, and the connection pool settings.

C. Session (Non-Thread-Safe)

The Session is a "lightweight" object that represents a single unit of work with the database. It wraps a JDBC connection. It is the primary interface used to perform CRUD operations.

Note: A Session should be opened and closed for every request/transaction; it should never be shared between threads.

D. Transaction

A Transaction represents a single atomic unit of work. In Hibernate, it is managed by the underlying database transaction or JTA (Java Transaction API). Always commit your transaction to ensure data is physically saved to the disk.

3. Annotations:

#@Entity and @Table

The @Entity annotation marks the class for Hibernate management. By default, the table name will match the class name. Use @Table to override this or to define unique constraints.

Example:

```
@Entity
```

```
@Table(name = "EMPLOYEES",  
uniqueConstraints{@UniqueConstraint(columnNames = "email")})
```

```
public class Employee { ... }
```

#Primary Key Mapping: @Id and @GeneratedValue

The @Id marks the identifier. The @GeneratedValue strategy determines how the ID is created:

- **AUTO:** Hibernate chooses the best strategy based on the database.
- **IDENTITY:** Uses the database's identity column (e.g., MySQL AUTO_INCREMENT).
- **SEQUENCE:** Uses a database sequence (common in Oracle/PostgreSQL).
- **TABLE:** Uses a separate table to keep track of IDs (slowest).

Example:

@Id

@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "emp_seq")

@SequenceGenerator(name = "emp_seq", sequenceName = "employee_sequence", allocationSize = 1)

private Long id;

#Attribute Mapping: @Column and @Lob

- **@Column:** Controls the mapping of a field to a column. nullable = false creates a NOT NULL constraint.
- **@Lob:** Indicates that the property should be persisted as a Large Object (BLOB for binary, CLOB for text).

#Relationship Mapping (The "Deep" Side)

One-To-Many and Many-To-One:

This is the most common relationship. Usually, the "Many" side is the **Owning Side** (the one that holds the foreign key).

Example:

// Inside Department.java

@OneToMany(mappedBy = "department") // 'mappedBy' makes this the inverse side

private List<Employee> employees;

// Inside Employee.java

@ManyToOne

```
@JoinColumn(name = "dept_id") // This creates the Foreign Key column
private Department department;
```

Many-To-Many:

This requires a **Join Table** to link the two entities.

Example:

```
@ManyToMany
```

```
@JoinTable( name = "student_course", joinColumns = @JoinColumn(name
= "student_id"), inverseJoinColumns = @JoinColumn(name = "course_id") )
```

```
private Set<Course> courses = new HashSet<>();
```

4. Hibernate States (Life Cycle)

Understanding the state of an object is critical for debugging:

1. **Transient:** The object is newly created using `new Employee()`. Hibernate does not know about it yet.
2. **Persistent:** The object is associated with a Session and has an ID in the database. Any changes made to this object will be automatically saved when the session is flushed.
3. **Detached:** The session is closed, but the object still exists. Hibernate is no longer tracking changes.
4. **Removed:** The object is scheduled to be deleted from the database.

5. Key Concepts:

#Caching Mechanics

- **First Level Cache:** Associated with the **Session**. It is mandatory. If you fetch the same object twice in the same session, Hibernate only hits the database once.
- **Second Level Cache:** Associated with the **SessionFactory**. It is optional (requires providers like Ehcache or Hazelcast). It stores data across different sessions.

#Fetching Strategies

- **FetchType.LAZY:** Data is loaded only when you call the getter (e.g., `dept.getEmployees()`). This prevents loading unnecessary data.
- **FetchType.EAGER:** Data is loaded immediately with a JOIN query. Use this carefully to avoid the **N+1 Select Problem**.

#The N+1 Select Problem

This occurs when you fetch a list of parent objects, and Hibernate executes 1 query to get the parents, and then N additional queries to get the children for each parent. To solve this, use **HQL "JOIN FETCH"** or **Entity Graphs**.

#Cascading Operations

The cascade attribute determines if an action (like save or delete) performed on a parent should automatically happen to the children.

- **CascadeType.ALL:** All operations are cascaded.
 - **CascadeType.PERSIST:** Only save operations are cascaded.
 - **orphanRemoval = true:** If a child is removed from a parent's collection, it is automatically deleted from the database.
-

#Spring Boot JDBC Template:

Spring JDBC Template is a powerful mechanism provided by the Spring Framework to simplify database operations. It handles the "boilerplate" code of traditional JDBC—such as opening/closing connections, handling exceptions, and managing statements—allowing you to focus strictly on writing SQL and mapping results.

1. Why JDBC Template over Plain JDBC?

Traditional JDBC requires a lot of repetitive code. For every query, you must:

1. Open a connection.
2. Create a statement.
3. Handle `SQLException` in a try-catch block.
4. Iterate through a `ResultSet`.
5. Close the connection, statement, and result set in a finally block.

Spring JDBC Template automates all of these steps. It uses the **Template Design Pattern**, where the framework handles the workflow and you provide the specific logic (SQL and Mapping).

2. Core Dependencies and Configuration

In Spring Boot, you only need the **Spring Boot Starter JDBC** dependency in your pom.xml:

<dependency>

groupId: org.springframework.boot

artifactId: spring-boot-starter-jdbc

</dependency>

Spring Boot automatically configures a JdbcTemplate bean for you as long as you provide the database credentials in your application.properties:

spring.datasource.url=jdbc:mysql://localhost:3306/mydb

spring.datasource.username=root spring.datasource.password=password

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

3. Key Methods in JdbcTemplate

The JdbcTemplate class provides several methods to execute SQL:

- **execute():** Generally used for DDL (Data Definition Language) queries like creating or dropping tables.
- **update():** Used for DML (Data Manipulation Language) queries like **INSERT, UPDATE, and DELETE**. It returns the number of rows affected.
- **queryForObject():** Used when the query is expected to return a **single row/result**.
- **query():** Used to fetch **multiple rows** and map them to a list of Java objects.

4. Data Mapping: RowMapper and ResultSetExtractor

Since SQL returns a ResultSet and Java uses Objects, we need a way to map them.

RowMapper (Most Common)

The `RowMapper<T>` interface is used to map each row of a `ResultSet` to a Java object.

Example:

```
public class StudentRowMapper implements RowMapper<Student>{

    @Override

    public Student mapRow(ResultSet rs, int rowNum) throws SQLException {

        Student student = new Student();

        student.setId(rs.getInt("id"));

        student.setName(rs.getString("name"));

        return student;

    }

}
```

BeanPropertyRowMapper

If your database column names match your Java variable names exactly, Spring provides a shortcut: `new BeanPropertyRowMapper<>(Student.class)`

5. Example:

The Repository Layer

In Spring Boot, we use the `@Repository` annotation on the class where we perform database operations. We use `@Autowired` to inject the `JdbcTemplate`.

`@Repository`

```
public class StudentRepository {

    @Autowired

    private JdbcTemplate jdbcTemplate;
```

// INSERT EXAMPLE

```
public int save(Student s) {  
  
    String sql = "INSERT INTO student (id, name) VALUES (?, ?)";  
  
    return jdbcTemplate.update(sql, s.getId(), s.getName());  
  
}
```

// SELECT SINGLE OBJECT

```
public Student findById(int id) {  
  
    String sql = "SELECT * FROM student WHERE id = ?";  
  
    return jdbcTemplate.queryForObject(sql, new  
BeanPropertyRowMapper<>(Student.class), id);  
  
}
```

// SELECT LIST OF OBJECTS

```
public List<Student> findAll() {  
  
    String sql = "SELECT * FROM student";  
  
    return jdbcTemplate.query(sql, (rs, rowNum) -> { Student s = new Student();  
s.setName(rs.getString("name"));  
  
return s; });  
  
}
```

6. Advanced Features:

- **NamedParameterJdbcTemplate**

Instead of using ? as placeholders (which can get confusing if you have 20 columns), you can use **Named Parameters** (e.g., :name).

Example: String sql = "INSERT INTO student (name) VALUES (:name)";
SqlParameterSource params = new MapSqlParameterSource("name",
student.getName()); namedParameterJdbcTemplate.update(sql, params);

- **Exception Handling**

Spring JDBC Template catches SQLException and translates it into a **DataAccessException**. This is a **Runtime (Unchecked) Exception**, so you aren't forced to write try-catch blocks everywhere, making your code much cleaner.

- **Batch Updates**

If you need to insert 1,000 records at once, using update() 1,000 times is slow. Use batchUpdate() to send them to the database in a single network call.

- **Callback Interfaces:**

- ❖ **PreparedStatementSetter:** Used to set values in a prepared statement manually.
- ❖ **ResultSetExtractor:** Used when you want to process the entire ResultSet at once (e.g., creating a complex Map) rather than row by row.